

Igor Klevanets <cerevra@yandex.ru>, <cerevra@yandex-team.ru>
Antony Polukhin <antoshkka@gmail.com>, <antoshkka@yandex-team.ru>

Date: 2017-01-28

A Proposal to add wide_int Template Class

I. Introduction and Motivation

Current standard provides signed and unsigned int8_t, int16_t, int32_t, int64_t. It is usually enough for every day tasks, but sometimes appears a need in big numbers: for cryptography, IPv6, very big counters etc. Non-standard type __int128 which is provided by gcc and clang illuminates this need. But there is no cross-platform solution and no way to satisfy future needs in even more big numbers.

This is an attempt to solve the problem in a generic way on a library level and provide wording for [P0104R0: Multi-Word Integer Operations and Types](#).

A proof of concept implementation available at: <https://github.com/cerevra/int/tree/master/v2>.

II. P0104R0 and this proposal

Differences with P0104R0:

- Template parameters were changed, from int words to size_t Bytes, bool Signed in attempt to make the class less platform dependent (avoid description of "machine word" term and it's size) and in attempt to minimize wording size by describing only one class, instead of two.
- Conversion to bool was made explicit to avoid user errors.
- Result of the arithmetic operations is a common_type_t of two types instead of a left-hand side type. This makes the wide_int class behave more close to the built-in types.

Extensions to P0104R0:

- Added operators to interact with built-in arithmetic types to simplify wide_int class usage.
- Added conversion operators to built-in arithmetic types to simplify usage of the wide_int class with existing code.
- Added literals, hashing, type_traits, numeric_limits and some functions to integrate better with existing code.

III. Proposed wording

26.??1 Header <wide_int> synopsis

[wide_int.syn]

```
namespace std {

    // 26.??2 class template wide_int
    template<size_t Bytes, bool Signed> class wide_int;

    // 26.??3 type traits specializations
    template<size_t Bytes, bool Signed> struct is_scalar<wide_int<Bytes, Signed>>: true_type {};
    template<size_t Bytes, bool Signed> struct is_compound<wide_int<Bytes, Signed>>: true_type {};
    template<size_t Bytes, bool Signed> struct is_unsigned<wide_int<Bytes, Signed>>: bool_constant<!Signed> {};
    template<size_t Bytes, bool Signed> struct is_signed<wide_int<Bytes, Signed>>: bool_constant<Signed> {};

    template<size_t Bytes, bool Signed, size_t Bytes2, bool Signed2>
    struct common_type<wide_int<Bytes, Signed>, wide_int<Bytes2, Signed2>>;

    template<size_t Bytes, bool Signed, typename Arithmetic>
    struct common_type<wide_int<Bytes, Signed>, Arithmetic>;

    // 26.??4 unary operations
    template<size_t Bytes, bool Signed> constexpr wide_int<Bytes, Signed> operator~(const wide_int<Bytes, Signed>& val) noexcept;
    template<size_t Bytes, bool Signed> constexpr wide_int<Bytes, Signed> operator-(const wide_int<Bytes, Signed>& val) noexcept(!Signed);
    template<size_t Bytes, bool Signed> constexpr wide_int<Bytes, Signed> operator+(const wide_int<Bytes, Signed>& val) noexcept(!Signed);

    // 26.??5 binary operations
    template<size_t Bytes, bool Signed, size_t Bytes2, bool Signed2>
    common_type_t<wide_int<Bytes, Signed>, wide_int<Bytes2, Signed2>>
    constexpr operator*(const wide_int<Bytes, Signed>& lhs, const wide_int<Bytes2, Signed2>& rhs);
    template<size_t Bytes, bool Signed, typename Arithmetic>
    common_type_t<wide_int<Bytes, Signed>, Arithmetic>
    constexpr operator*(const wide_int<Bytes, Signed>& lhs, const Arithmetic& rhs);
    template<size_t Bytes, bool Signed, typename Arithmetic>
    common_type_t<wide_int<Bytes, Signed>, Arithmetic>
    constexpr operator*(const Arithmetic& lhs, const wide_int<Bytes, Signed>& rhs);

    template<size_t Bytes, bool Signed, size_t Bytes2, bool Signed2>
    common_type_t<wide_int<Bytes, Signed>, wide_int<Bytes2, Signed2>>
    constexpr operator/(const wide_int<Bytes, Signed>& lhs, const wide_int<Bytes2, Signed2>& rhs);
```

[illegible]

}

width.

]

[wide int.overview]

```
namespace std {
    template<size_t Bytes, bool Signed>
    class wide_int {
    public:
        // 26.??2.?? construct:
        constexpr wide_int() noexcept = default;
        template<typename Arithmetic> constexpr wide_int(const Arithmetic& other) noexcept;
        template<size_t Bytes2, bool Signed2> constexpr wide_int(const wide_int<Bytes2, Signed2>& other) noexcept;

        // 26.??2.?? assignment:
        template<typename Arithmetic>
        constexpr wide_int<Bytes, Signed> & operator=(const Arithmetic& other) noexcept;
        template<size_t Bytes2, bool Signed2>
        constexpr wide_int<Bytes, Signed> & operator=(const wide_int<Bytes2, Signed2>& other) noexcept;

        // 26.??2.?? compound assignment:
        template<typename Arithmetic>
        constexpr wide_int<Bytes, Signed> & operator*=(const Arithmetic&);
        template<size_t Bytes2, bool Signed2>
        constexpr wide_int<Bytes, Signed> & operator*=(const wide_int<Bytes2, Signed2>&);
    };
}
```

```

template<typename Arithmetic>
constexpr wide_int<Bytes, Signed>& operator/=(const Arithmetic&);
template<size_t Bytes2, bool Signed2>
constexpr wide_int<Bytes, Signed>& operator/=(const wide_int<Bytes2, Signed2>&);

template<typename Arithmetic>
constexpr wide_int<Bytes, Signed>& operator+=(const Arithmetic&) noexcept(!Signed);
template<size_t Bytes2, bool Signed2>
constexpr wide_int<Bytes, Signed>& operator+=(const wide_int<Bytes2, Signed2>&) noexcept(!Signed);

template<typename Arithmetic>
constexpr wide_int<Bytes, Signed>& operator-=(const Arithmetic&) noexcept(!Signed);
template<size_t Bytes2, bool Signed2>
constexpr wide_int<Bytes, Signed>& operator-=(const wide_int<Bytes2, Signed2>&) noexcept(!Signed);

template<typename Integral>
constexpr wide_int<Bytes, Signed>& operator%=(const Integral&);
template<size_t Bytes2, bool Signed2>
constexpr wide_int<Bytes, Signed>& operator%=(const wide_int<Bytes2, Signed2>&);

template<typename Integral>
constexpr wide_int<Bytes, Signed>& operator&=(const Integral&) noexcept;
template<size_t Bytes2, bool Signed2>
constexpr wide_int<Bytes, Signed>& operator&=(const wide_int<Bytes2, Signed2>&) noexcept;

template<typename Integral>
constexpr wide_int<Bytes, Signed>& operator|=(const Integral&) noexcept;
template<size_t Bytes2, bool Signed2>
constexpr wide_int<Bytes, Signed>& operator|=(const wide_int<Bytes2, Signed2>&) noexcept;

template<typename Integral>
constexpr wide_int<Bytes, Signed>& operator^=(const Integral&) noexcept;
template<size_t Bytes2, bool Signed2>
constexpr wide_int<Bytes, Signed>& operator^=(const wide_int<Bytes2, Signed2>&) noexcept;

template<typename Integral>
constexpr wide_int<Bytes, Signed>& operator<=<=(const Integral&);
template<size_t Bytes2, bool Signed2>
constexpr wide_int<Bytes, Signed>& operator<=<=(const wide_int<Bytes2, Signed2>&);

template<typename Integral>
constexpr wide_int<Bytes, Signed>& operator>=>=(const Integral&) noexcept;
template<size_t Bytes2, bool Signed2>
constexpr wide_int<Bytes, Signed>& operator>=>=(const wide_int<Bytes2, Signed2>&) noexcept;

// 26.???.2.?? observers:
template<typename Arithmetic> constexpr operator Arithmetic() const noexcept;
constexpr explicit operator bool() const noexcept;

private:
    uint32_t data[Bytes / sizeof(uint32_t) + 1]; // exposition only
};
}

```

The class template `wide_int<size_t Bytes, bool Signed>` is a POD class that behaves as an integer type of a compile time specified width. Template parameter `Bytes` specifies significant bytes count to store the integer value. *[Note: `sizeof(wide_int<Bytes, true>)` and `sizeof(wide_int<Bytes, false>)` are not required to be equal to `Bytes`. - end note]* Template parameter `Signed` specifies signedness of the stored integer value.

26.???.2.?? wide_int constructors

[wide_int.cons]

```
constexpr wide_int() noexcept = default;
```

Effects: Constructs an object with undefined value.

```
template<typename Arithmetic> constexpr wide_int(const Arithmetic& other) noexcept;
```

Remarks: This function shall not participate in overload resolution unless `is_arithmetic_v<Arithmetic>` is true.

Effects: Constructs an object from `other` using the integral conversion rules [conv.integral].

```
template<size_t Bytes2, bool Signed2> constexpr wide_int(const wide_int<Bytes2, Signed2>& other) noexcept;
```

Effects: Constructs an object from `other` using the integral conversion rules [conv.integral].

26.???.2.?? wide_int assignments

[wide_int.assign]

```
template<typename Arithmetic>
constexpr wide_int<Bytes, Signed>& operator=(const Arithmetic& other) noexcept;
```

Remarks: This function shall not participate in overload resolution unless `is_arithmetic_v<Arithmetic>` is true.

Effects: Constructs an object from `other` using the integral conversion rules [conv.integral].

```
template<size_t Bytes2, bool Signed2>
constexpr wide_int<Bytes, Signed>& operator=(const wide_int<Bytes2, Signed2>& other) noexcept;
```

Effects: Constructs an object from `other` using the integral conversion rules [conv.integral].

26.???.2.?? wide_int compound assignments

[wide_int.cassign]

```
template<size_t Bytes2, bool Signed2> constexpr wide_int<Bytes, Signed>& operator*=(const wide_int<Bytes2, Signed2>&);
template<size_t Bytes2, bool Signed2> constexpr wide_int<Bytes, Signed>& operator/=(const wide_int<Bytes2, Signed2>&);
template<size_t Bytes2, bool Signed2> constexpr wide_int<Bytes, Signed>& operator+=(const wide_int<Bytes2, Signed2>&) noexcept(!Signed);
template<size_t Bytes2, bool Signed2> constexpr wide_int<Bytes, Signed>& operator-=(const wide_int<Bytes2, Signed2>&) noexcept(!Signed);
template<size_t Bytes2, bool Signed2> constexpr wide_int<Bytes, Signed>& operator%=(const wide_int<Bytes2, Signed2>&);
template<size_t Bytes2, bool Signed2> constexpr wide_int<Bytes, Signed>& operator&=(const wide_int<Bytes2, Signed2>&) noexcept;
template<size_t Bytes2, bool Signed2> constexpr wide_int<Bytes, Signed>& operator|=(const wide_int<Bytes2, Signed2>&) noexcept;
template<size_t Bytes2, bool Signed2> constexpr wide_int<Bytes, Signed>& operator^=(const wide_int<Bytes2, Signed2>&) noexcept;
template<size_t Bytes2, bool Signed2> constexpr wide_int<Bytes, Signed>& operator<=(const wide_int<Bytes2, Signed2>&);
template<size_t Bytes2, bool Signed2> constexpr wide_int<Bytes, Signed>& operator>=(const wide_int<Bytes2, Signed2>&) noexcept;
```

Effects: Behavior of these operators is similar to operators for built-in integral types.

```
template<typename Arithmetic> constexpr wide_int<Bytes, Signed>& operator*=(const Arithmetic&);
template<typename Arithmetic> constexpr wide_int<Bytes, Signed>& operator/=(const Arithmetic&);
template<typename Arithmetic> constexpr wide_int<Bytes, Signed>& operator+=(const Arithmetic&) noexcept(!Signed);
template<typename Arithmetic> constexpr wide_int<Bytes, Signed>& operator-=(const Arithmetic&) noexcept(!Signed);
```

Remarks: Functions shall not participate in overload resolution unless `is_arithmetic_v<Arithmetic>` is true.

Effects: As if an object `wi` of type `wide_int<Bytes, Signed>` was created from input value and the corresponding operator was called for `*this` and the `wi`.

```
template<typename Integral> constexpr wide_int<Bytes, Signed>& operator%=(const Integral&);
template<typename Integral> constexpr wide_int<Bytes, Signed>& operator&=(const Integral&) noexcept;
template<typename Integral> constexpr wide_int<Bytes, Signed>& operator|=(const Integral&) noexcept;
template<typename Integral> constexpr wide_int<Bytes, Signed>& operator^=(const Integral&) noexcept;
template<typename Integral> constexpr wide_int<Bytes, Signed>& operator<=(const Integral&);
template<typename Integral> constexpr wide_int<Bytes, Signed>& operator>=(const Integral&) noexcept;
```

Remarks: Functions shall not participate in overload resolution unless `is_integral_v<Integral>` is true.

Effects: As if an object `wi` of type `wide_int<Bytes, Signed>` was created from input value and the corresponding operator was called for `*this` and the `wi`.

26.???.?? wide_int observers

[wide_int.observers]

```
template <typename Arithmetic> constexpr operator Arithmetic() const noexcept;
```

Remarks: This function shall not participate in overload resolution unless `is_arithmetic_v<Arithmetic>` is true.

Returns: If `Arithmetic` type is an integral type then it is constructed from `*this` using the integral conversion rules [conv.integral]. Otherwise `Arithmetic` is constructed from `*this` using the floating-integral conversion rules [conv.fpint].

```
constexpr explicit operator bool() const noexcept;
```

Returns: true if `*this` is not equal to 0.

26.???.?? Specializations of common_type

[wide_int.traits.specializations]

```
template<size_t Bytes, bool Signed, size_t Bytes2, bool Signed2>
struct common_type<wide_int<Bytes, Signed>, wide_int<Bytes2, Signed2>> {
    using type = wide_int<max(Bytes, Bytes2), see below>;
};
```

The signed template parameter indicated by this specialization is following:

- (Signed && Signed2) if Bytes == Bytes2
- Signed if Bytes > Bytes2
- Signed2 otherwise

```
template<size_t Bytes, bool Signed, typename Arithmetic>
struct common_type<wide_int<Bytes, Signed>, Arithmetic> {
    using type = see below;
};
```

The member typedef type is following:

- Arithmetic if `is_floating_point_v<Arithmetic>` is true
- `wide_int<Bytes, Signed>` if Bytes > sizeof(Arithmetic)
- Arithmetic if Bytes < sizeof(Arithmetic)
- Arithmetic if Bytes == sizeof(Arithmetic) && Signed OR Bytes == sizeof(Arithmetic) && Signed == `is_signed_v<Arithmetic>`
- `wide_int<Bytes, Signed>` otherwise

Requires: `is_arithmetic_v<Arithmetic>` is true.

26.???.?? Unary operators

[wide_int.unary_ops]

```
template<size_t Bytes, bool Signed> constexpr wide_int<Bytes, Signed> operator~(const wide_int<Bytes, Signed>& val) noexcept;
```

Returns: value with inverted significant bits of `val`.

```
template<size_t Bytes, bool Signed> constexpr wide_int<Bytes, Signed> operator-(const wide_int<Bytes, Signed>& val) noexcept(!Signed);
```


Returns: val *= -1 if Signed is true, otherwise the result is unspecified.

```
template<size_t Bytes, bool Signed> constexpr wide_int<Bytes, Signed> operator+(const wide_int<Bytes, Signed>& val) noexcept(!Signed);
```

Returns: val.

26.???.? Binary operators

[wide_int.binary_ops]

In the function descriptions that follow, CT represents common_type_t<A, B>, where A and B are the types of the two arguments to the function.

Functions that accept Integral parameter shall not participate in overload resolution unless is_integral_v<Integral> is true.

Functions that accept Arithmetic parameter shall not participate in overload resolution unless is_arithmetic_v<Arithmetic> is true.

[Note: To reduce template instantiations count operators that accept Integral and Arithmetic parameter convert to common type first, as the common type may be a built-in type. - end note]

```
template<size_t Bytes, bool Signed, size_t Bytes2, bool Signed2>
common_type_t<wide_int<Bytes, Signed>, wide_int<Bytes2, Signed2>>
constexpr operator*(const wide_int<Bytes, Signed>& lhs, const wide_int<Bytes2, Signed2>& rhs);
```

Returns: CT(lhs) *= rhs.

```
template<size_t Bytes, bool Signed, typename Arithmetic>
common_type_t<wide_int<Bytes, Signed>, Arithmetic>
constexpr operator*(const wide_int<Bytes, Signed>& lhs, const Arithmetic& rhs);
template<size_t Bytes, bool Signed, typename Arithmetic>
common_type_t<wide_int<Bytes, Signed>, Arithmetic>
constexpr operator*(const Arithmetic& lhs, const wide_int<Bytes, Signed>& rhs);
```

Returns: CT(lhs) * CT(rhs).

```
template<size_t Bytes, bool Signed, size_t Bytes2, bool Signed2>
common_type_t<wide_int<Bytes, Signed>, wide_int<Bytes2, Signed2>>
constexpr operator/(const wide_int<Bytes, Signed>& lhs, const wide_int<Bytes2, Signed2>& rhs);
```

Returns: CT(lhs) /= rhs.

```
template<size_t Bytes, bool Signed, typename Arithmetic>
common_type_t<wide_int<Bytes, Signed>, Arithmetic>
constexpr operator/(const wide_int<Bytes, Signed>& lhs, const Arithmetic& rhs);
template<size_t Bytes, bool Signed, typename Arithmetic>
common_type_t<wide_int<Bytes, Signed>, Arithmetic>
constexpr operator/(const Arithmetic& lhs, const wide_int<Bytes, Signed>& rhs);
```

Returns: CT(lhs) / CT(rhs).

```
template<size_t Bytes, bool Signed, size_t Bytes2, bool Signed2>
common_type_t<wide_int<Bytes, Signed>, wide_int<Bytes2, Signed2>>
constexpr operator+(const wide_int<Bytes, Signed>& lhs, const wide_int<Bytes2, Signed2>& rhs) noexcept(!Signed);
```

Returns: CT(lhs) += rhs.

```
template<size_t Bytes, bool Signed, typename Arithmetic>
common_type_t<wide_int<Bytes, Signed>, Arithmetic>
constexpr operator+(const wide_int<Bytes, Signed>& lhs, const Arithmetic& rhs) noexcept(!Signed);
template<size_t Bytes, bool Signed, typename Arithmetic>
common_type_t<wide_int<Bytes, Signed>, Arithmetic>
constexpr operator+(const Arithmetic& lhs, const wide_int<Bytes, Signed>& rhs) noexcept(!Signed);
```

Returns: CT(lhs) + CT(rhs).

```
template<size_t Bytes, bool Signed, size_t Bytes2, bool Signed2>
common_type_t<wide_int<Bytes, Signed>, wide_int<Bytes2, Signed2>>
constexpr operator-(const wide_int<Bytes, Signed>& lhs, const wide_int<Bytes2, Signed2>& rhs) noexcept(!Signed);
```

Returns: CT(lhs) -= rhs.

```
template<size_t Bytes, bool Signed, typename Arithmetic>
common_type_t<wide_int<Bytes, Signed>, Arithmetic>
constexpr operator-(const wide_int<Bytes, Signed>& lhs, const Arithmetic& rhs) noexcept(!Signed);
template<size_t Bytes, bool Signed, typename Arithmetic>
common_type_t<wide_int<Bytes, Signed>, Arithmetic>
constexpr operator-(const Arithmetic& lhs, const wide_int<Bytes, Signed>& rhs) noexcept(!Signed);
```

Returns: CT(lhs) - CT(rhs).

```
template<size_t Bytes, bool Signed, size_t Bytes2, bool Signed2>
common_type_t<wide_int<Bytes, Signed>, wide_int<Bytes2, Signed2>>
constexpr operator%(const wide_int<Bytes, Signed>& lhs, const wide_int<Bytes2, Signed2>& rhs);
```

Returns: CT(lhs) %= rhs.

```
template<size_t Bytes, bool Signed, typename Integral>
common_type_t<wide_int<Bytes, Signed>, Integral>
```

```
constexpr operator%(const wide_int<Bytes, Signed>& lhs, const Integral& rhs);
template<size_t Bytes, bool Signed, typename Integral>
common_type_t<wide_int<Bytes, Signed>, Integral>
constexpr operator%(const Integral& lhs, const wide_int<Bytes, Signed>& rhs);
```

Returns: CT(lhs) % CT(rhs).

```
template<size_t Bytes, bool Signed, size_t Bytes2, bool Signed2>
common_type_t<wide_int<Bytes, Signed>, wide_int<Bytes2, Signed2>>
constexpr operator&(const wide_int<Bytes, Signed>& lhs, const wide_int<Bytes2, Signed2>& rhs) noexcept;
```

Returns: CT(lhs) &= rhs.

```
template<size_t Bytes, bool Signed, typename Integral>
common_type_t<wide_int<Bytes, Signed>, Integral>
constexpr operator&(const wide_int<Bytes, Signed>& lhs, const Integral& rhs) noexcept;
template<size_t Bytes, bool Signed, typename Integral>
common_type_t<wide_int<Bytes, Signed>, Integral>
constexpr operator&(const Integral& lhs, const wide_int<Bytes, Signed>& rhs) noexcept;
```

Returns: CT(lhs) & CT(rhs).

```
template<size_t Bytes, bool Signed, size_t Bytes2, bool Signed2>
common_type_t<wide_int<Bytes, Signed>, wide_int<Bytes2, Signed2>>
constexpr operator| (const wide_int<Bytes, Signed>& lhs, const wide_int<Bytes2, Signed2>& rhs) noexcept;
```

Returns: CT(lhs) |= rhs.

```
template<size_t Bytes, bool Signed, typename Integral>
common_type_t<wide_int<Bytes, Signed>, Integral>
constexpr operator|(const wide_int<Bytes, Signed>& lhs, const Integral& rhs) noexcept;
template<size_t Bytes, bool Signed, typename Integral>
common_type_t<wide_int<Bytes, Signed>, Integral>
constexpr operator|(const Integral& lhs, const wide_int<Bytes, Signed>& rhs) noexcept;
```

Returns: CT(lhs) | CT(rhs).

```
template<size_t Bytes, bool Signed, size_t Bytes2, bool Signed2>
common_type_t<wide_int<Bytes, Signed>, wide_int<Bytes2, Signed2>>
constexpr operator^(const wide_int<Bytes, Signed>& lhs, const wide_int<Bytes2, Signed2>& rhs) noexcept;
```

Returns: CT(lhs) ^= rhs.

```
template<size_t Bytes, bool Signed, typename Integral>
common_type_t<wide_int<Bytes, Signed>, Integral>
constexpr operator^(const wide_int<Bytes, Signed>& lhs, const Integral& rhs) noexcept;
template<size_t Bytes, bool Signed, typename Integral>
common_type_t<wide_int<Bytes, Signed>, Integral>
constexpr operator^(const Integral& lhs, const wide_int<Bytes, Signed>& rhs) noexcept;
```

Returns: CT(lhs) ^ CT(rhs).

```
template<size_t Bytes, bool Signed, size_t Bytes2, bool Signed2>
common_type_t<wide_int<Bytes, Signed>, wide_int<Bytes2, Signed2>>
constexpr operator<<(const wide_int<Bytes, Signed>& lhs, const wide_int<Bytes2, Signed2>& rhs);
```

Returns: CT(lhs) <<= rhs.

```
template<size_t Bytes, bool Signed, typename Integral>
constexpr wide_int<Bytes, Signed> operator<<(const wide_int<Bytes, Signed>& lhs, const Integral& rhs);
```

Returns: CT(lhs) << CT(rhs).

```
template<size_t Bytes, bool Signed, typename Integral>
constexpr Integral operator<<(const Integral& lhs, const wide_int<Bytes, Signed>& rhs);
```

Returns: lhs << Integral(rhs).

```
template<size_t Bytes, bool Signed, size_t Bytes2, bool Signed2>
common_type_t<wide_int<Bytes, Signed>, wide_int<Bytes2, Signed2>>
constexpr operator>>(const wide_int<Bytes, Signed>& lhs, const wide_int<Bytes2, Signed2>& rhs) noexcept;
```

Returns: CT(lhs) >>= rhs.

```
template<size_t Bytes, bool Signed, typename Integral>
constexpr wide_int<Bytes, Signed> operator>>(const wide_int<Bytes, Signed>& lhs, const Integral& rhs) noexcept;
```

Returns: CT(lhs) >> CT(rhs).

```
template<size_t Bytes, bool Signed, typename Integral>
constexpr Integral operator>>(const Integral& lhs, const wide_int<Bytes, Signed>& rhs) noexcept;
```

Returns: lhs >> Integral(rhs).

```
template<size_t Bytes, bool Signed, size_t Bytes2, bool Signed2>
constexpr bool operator<(const wide_int<Bytes, Signed>& lhs, const wide_int<Bytes2, Signed2>& rhs) noexcept;
```

Returns: true if value of CT(lhs) is less than the value of CT(rhs).

```
template<size_t Bytes, bool Signed, typename Arithmetic>
constexpr bool operator< (const wide_int<Bytes, Signed>& lhs, const Arithmetic& rhs) noexcept;
template<size_t Bytes, bool Signed, typename Arithmetic>
constexpr bool operator< (const Arithmetic& lhs, const wide_int<Bytes, Signed>& rhs) noexcept;
```

Returns: CT(lhs) < CT(rhs).

```
template<size_t Bytes, bool Signed, size_t Bytes2, bool Signed2>
constexpr bool operator> (const wide_int<Bytes, Signed>& lhs, const wide_int<Bytes2, Signed2>& rhs) noexcept;
```

Returns: true if value of CT(lhs) is greater than the value of CT(rhs).

```
template<size_t Bytes, bool Signed, typename Arithmetic>
constexpr bool operator> (const wide_int<Bytes, Signed>& lhs, const Arithmetic& rhs) noexcept;
template<size_t Bytes, bool Signed, typename Arithmetic>
constexpr bool operator> (const Arithmetic& lhs, const wide_int<Bytes, Signed>& rhs) noexcept;
```

Returns: CT(lhs) > CT(rhs).

```
template<size_t Bytes, bool Signed, size_t Bytes2, bool Signed2>
constexpr bool operator<= (const wide_int<Bytes, Signed>& lhs, const wide_int<Bytes2, Signed2>& rhs) noexcept;
```

Returns: true if value of CT(lhs) is equal or less than the value of CT(rhs).

```
template<size_t Bytes, bool Signed, typename Arithmetic>
constexpr bool operator<= (const wide_int<Bytes, Signed>& lhs, const Arithmetic& rhs) noexcept;
template<size_t Bytes, bool Signed, typename Arithmetic>
constexpr bool operator<= (const Arithmetic& lhs, const wide_int<Bytes, Signed>& rhs) noexcept;
```

Returns: CT(lhs) <= CT(rhs).

```
template<size_t Bytes, bool Signed, size_t Bytes2, bool Signed2>
constexpr bool operator>= (const wide_int<Bytes, Signed>& lhs, const wide_int<Bytes2, Signed2>& rhs) noexcept;
```

Returns: true if value of CT(lhs) is equal or greater than the value of CT(rhs).

```
template<size_t Bytes, bool Signed, typename Arithmetic>
constexpr bool operator>= (const wide_int<Bytes, Signed>& lhs, const Arithmetic& rhs) noexcept;
template<size_t Bytes, bool Signed, typename Arithmetic>
constexpr bool operator>= (const Arithmetic& lhs, const wide_int<Bytes, Signed>& rhs) noexcept;
```

Returns: CT(lhs) >= CT(rhs).

```
template<size_t Bytes, bool Signed, size_t Bytes2, bool Signed2>
constexpr bool operator== (const wide_int<Bytes, Signed>& lhs, const wide_int<Bytes2, Signed2>& rhs) noexcept;
```

Returns: true if significant bits of CT(lhs) and CT(rhs) are the same.

```
template<size_t Bytes, bool Signed, typename Arithmetic>
constexpr bool operator== (const wide_int<Bytes, Signed>& lhs, const Arithmetic& rhs) noexcept;
template<size_t Bytes, bool Signed, typename Arithmetic>
constexpr bool operator== (const Arithmetic& lhs, const wide_int<Bytes, Signed>& rhs) noexcept;
```

Returns: CT(lhs) == CT(rhs).

```
template<size_t Bytes, bool Signed, size_t Bytes2, bool Signed2>
constexpr bool operator!= (const wide_int<Bytes, Signed>& lhs, const wide_int<Bytes2, Signed2>& rhs) noexcept;
```

Returns: !(CT(lhs) == CT(rhs)).

```
template<size_t Bytes, bool Signed, typename Arithmetic>
constexpr bool operator!= (const wide_int<Bytes, Signed>& lhs, const Arithmetic& rhs) noexcept;
template<size_t Bytes, bool Signed, typename Arithmetic>
constexpr bool operator!= (const Arithmetic& lhs, const wide_int<Bytes, Signed>& rhs) noexcept;
```

Returns: CT(lhs) != CT(rhs).

26.???.? Numeric conversions

[wide_int.conversions]

```
template<size_t Bytes, bool Signed> std::string to_string(const wide_int<Bytes, Signed>& val);
template<size_t Bytes, bool Signed> std::wstring to_wstring(const wide_int<Bytes, Signed>& val);
```

Returns: Each function returns an object holding the character representation of the value of its argument. All the significant bits of the argument are outputted as a signed decimal in the style [-]dddd.

26.???.? iostream specializations

[wide_int.io]

```
template<class Char, class Traits, size_t Bytes, bool Signed>
basic_ostream<Char, Traits>& operator<< (basic_ostream<Char, Traits>& os, const wide_int<Bytes, Signed>& val);
```

Effects: As if by: os << to_string(val).

Returns: os.

```
template<class Char, class Traits, size_t Bytes, bool Signed>
basic_istream<Char, Traits>& operator>> (basic_istream<Char, Traits>& is, wide_int<Bytes, Signed>& val);
```


Effects: Extracts a `wide_int` that is represented as a decimal number in the `is`. If bad input is encountered, calls `is.setstate(ios_base::failbit)` (which may throw `ios::failure` (`!istate.flags`)).

Returns: `is`.

26.???.? Hash support

[wide_int.hash]

```
template<size_t Bytes, bool Signed> struct hash<wide_int<Bytes, Signed>>;
```

The specialization is enabled (20.14.14). If there is a built-in integral type `Integral` that has the same signedness and width as `wide_int<Bytes, Signed>`, and `wi` is an object of type `wide_int<Bytes, Signed>`, then `hash<wide_int<Bytes, Signed>>(wi) == hash<Integral>(Integral(wi))`.

26.???.? numeric_limits specialization

[wide_int.numeric_limits]

```
template<size_t Bytes, bool Signed> struct numeric_limits<wide_int<Bytes, Signed>>;
```

Specialization follows the rules described in `[numeric_limits.members]`. If there is a built-in integral type `Integral` that has the same signedness and width as `wide_int<Bytes, Signed>`, then `numeric_limits<wide_int<Bytes, Signed>>` specialized in the same way as `numeric_limits<Integral>`.

IV. Feature-testing macro

For the purposes of SG10 it is sufficient to check for header `<wide_int>` using `__has_include`.